Chapter 01 -- Entering the World of Ontology Engineering with Protégé and `Pizza.owl`

The journey into ontology engineering often begins with a deceptively simple example: `pizza`. Behind this famous `pizza.owl` tutorial lies one of the most influential learning paths in the Semantic Web and Knowledge Graph ecosystem. In this first chapter, we establish the conceptual foundations of ontology modeling, introduce the Protégé ontology editor, and position ontology engineering within the broader vision of Executable Knowledge Architecture (EKA).

This chapter is based primarily on Michael DeBellis' excellent tutorial *A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins*, which modernized and extended the original Manchester Pizza Tutorial into a comprehensive hands-on learning experience.

At the same time, this book extends beyond the tutorial itself. The goal here is not only to learn Protégé as a standalone modeling tool, but to understand ontology as a critical layer in the transformation from diagrams to executable intelligence.

- 1.1 Why the Pizza Tutorial Became an Industry Classic
- 1.2 Protégé: More Than an Ontology Editor
- 1.3 The EKA Perspective: From Diagrams to Executable Intelligence
- 1.4 Understanding OWL: The Semantic Core
  - 1.4.1 Classes: Modeling Conceptual Categories
  - 1.4.2 Individuals: Representing Concrete Instances
  - 1.4.3 Properties: Defining Relationships
- 1.5 Ontology vs Traditional Data Modeling
- 1.6 Open World Assumption: A New Way of Thinking
- 1.7 Reasoners: The Engine Behind Semantic Intelligence
- 1.8 Why Ontology Matters in the AI Era
- 1.9 What You Will Build Throughout This Book
- 1.10 The Bigger Picture
- 1.11 Chapter 01 Summary
- 1.12 Next Chapter Preview
- 1.13 Reference
- 1.14 Key Concepts
- 1.15 One More Thing...

1.1 Why the Pizza Tutorial Became an Industry Classic

The `pizza.owl` tutorial has become the "Hello World" of ontology engineering for several reasons.

First, pizza is universally understandable. Nearly everyone intuitively understands toppings, ingredients, vegetarian classifications, spicy foods, and regional styles. This makes it easier to focus on semantic modeling

concepts rather than domain complexity.

Second, the tutorial gradually introduces increasingly sophisticated OWL concepts in a controlled environment. Instead of overwhelming learners with abstract logic theory, it teaches ontology construction incrementally:

- Classes
- Subclasses
- Individuals
- Object Properties
- Restrictions
- Reasoners
- Logical Inference
- SWRL Rules
- SPARQL Queries
- SHACL Validation

This progression mirrors how real enterprise ontology projects evolve.

Third, the tutorial demonstrates something fundamentally important:

> Ontologies are not just diagrams.
>
> They are executable semantic systems.

This is the core idea that later becomes central in EKA.

1.2 Protégé: More Than an Ontology Editor

Protégé is often introduced simply as an ontology editor, but this description dramatically understates its importance.

Protégé is better understood as a **semantic modeling platform**.

It allows users to:

- Define domain concepts
- Encode formal semantics
- Build machine-readable knowledge structures
- Execute logical reasoning
- Validate constraints
- Query semantic relationships
- Prepare foundations for knowledge graphs

In traditional enterprise architecture tools, architects often create static diagrams that describe systems visually but lack executable semantics.

For example:

- UML diagrams describe structure
- BPMN diagrams describe processes
- ER diagrams describe data
- Capability maps describe organizations

However, these artifacts usually remain disconnected documentation.

Protégé introduces a fundamentally different paradigm.

Instead of merely drawing concepts, we formally define them using **logic-based semantics**. The ontology becomes interpretable by machines, enabling automated reasoning and inference.

This transition is foundational for modern AI-native architectures.

1.3 The EKA Perspective: From Diagrams to Executable Intelligence

A slogan now circulating in industrial semantics is *Meaning before Automation*, associated with Frédéric Verhelst's *The Ontology Imperative*. At PLM Road May & PDT EMEA, one panellist put the underlying compliant plainly: his PLM vendors give him no ontology definitions, and where an ontology appears at all it comes as an afterthought, added at the end to build a knowledge graph. He wants it as the foundation -- "the agreed contract between systems." That compliant is the practical meaning of the slogan: the ontology is not an accessory to automation but its precondition. EKA formalizes that contract as an executable knowledge architecture, and the `Pizza` ontology is the first place we build one end to end.

Within the EKA (Executable Knowledge Architecture) framework introduced on xiaoqi.com, ontology engineering occupies a critical transformation layer.

The implementation roadmap of EKA can be summarized as:



Graph Database Loading Paths for OWL Ontologies

OWl ontologies can be loaded into graph databases, but the implementation path depends on how much semantic fidelity you need to preserve:

1. **Native RDF Graph Databases (Recommended for EKA)**: AllegroGraph, GraphDB, Stardog, RDFox, Amazon Neptune (RDF mode), and Apache Jena/Fuseki. These systems store data natively as RDF triples and can **load OWL ontologies directly**, preserving full formal semantics, global IRI identity, and reasoning capabilities. This is the recommended choice for the EKA K layer.
2. **Property Graphs (For Specialized Performance Scenarios)**: Neo4j, TigerGraph, JanusGraph, and Amazon Neptune (property graph mode). These systems use internal, non-semantic element IDs. Loading an OWL ontology into a property graph is a **lossy transformation**, not a direct mapping. You

> lose OWL semantics, reasoning capabilities, and IRI-based global interoperability. Use this path only for specialized, performance-intensive graph traversal scenarios.

The Pizza OWL tutorial specifically addresses the "Ontology" stage.

To understand why this matters, we must examine the limitations of traditional enterprise modeling.

Most enterprise architecture repositories contain thousands of diagrams:

- Application landscapes
- Process flows
- Integration maps
- Data models
- Organizational charts

But these assets - very valuable - are typically:

- Static
- Human-readable only
- Semantically ambiguous
- Non-executable
- Difficult to integrate

Ontology changes this situation fundamentally.

When enterprise concepts become ontological entities:

- Relationships gain formal meaning
- Constraints become machine-processable
- Inference becomes possible
- Knowledge graphs can be generated
- AI systems can consume structured semantics

In EKA terminology, ontology acts as the semantic bridge between abstract architecture and executable intelligence.

This is why learning Protégé is not merely learning another modeling tool.

IT IS learning how to formalize knowledge itself.

1.4 Understanding OWL: The Semantic Core

The tutorial centers around OWL, the Web Ontology Language.

Web Ontology Language is a W3C standard designed to represent knowledge in a machine-readable and logically rigorous form.

Unlike relational databases, OWL is not primarily designed for transaction processing.

Instead, OWL focuses on:

- Meaning
- Classification
- Inference
- Semantic relationships
- Knowledge interoperability

Michael DeBellis emphasizes that OWL ontologies are composed primarily of three core elements:

1. Classes
2. Individuals
3. Properties

These concepts form the semantic building blocks of all ontology systems.

1.4.1 Classes: Modeling Conceptual Categories

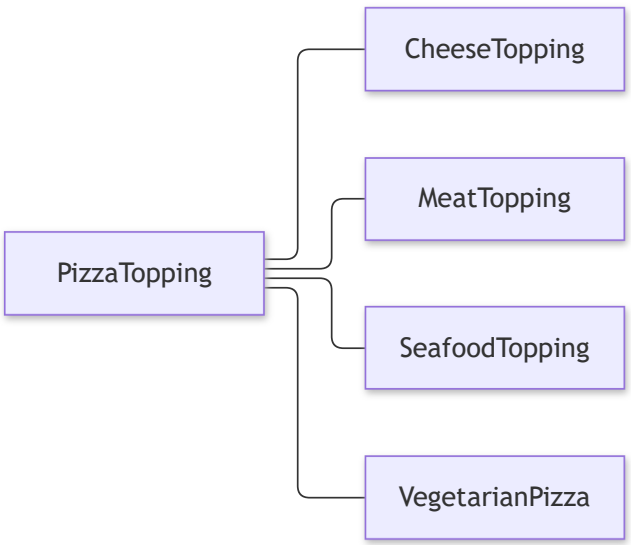
A class represents a category or type of thing.

Examples in the Pizza ontology include:

- Pizza
- PizzaTopping
- CheeseTopping
- VegetableTopping
- VegetarianPizza

Classes can be organized hierarchically.

For example:



You may also document above graph in this way:

```
PizzaTopping
 |-- CheeseTopping
 |-- MeatTopping
 |-- SeafoodTopping
 |-- VegetableTopping
```

This resembles inheritance structure in object-oriented modeling, but OWL classes are fundamentally semantic rather than implementation-oriented.

A class in OWL describes a set of possible individuals.

This distinction becomes important later when reasoning engines classify instances automatically.

1.4.2 Individuals: Representing Concrete Instances

Individuals are actual members of classes.

For example, members of class `Pizza` or `PizzaTopping` can be:

- MargheritaPizza
- MozzarellaTopping
- SpicyBeefPizza

An individual belongs to one or more classes.

However, unlike traditional systems where classification is often manually assigned, OWL reasoners can infer class membership automatically based on logical definitions.

This capability is one of the major breakthroughs of ontology systems.

1.4.3 Properties: Defining Relationships

Properties define how entities relate to one another.

In OWL, there are two major property types:

1. Object Properties: These connect individuals to other individuals.

Example:

```
hasTopping
```

2. Data Properties: These connect individuals to literal value.

Example:

```
hasCalories
hasPrice
hasSpicinessLevel
```

Properties become extraordinarily powerful when combined with restrictions and logical constraints.

1.5 Ontology vs Traditional Data Modeling

New learners often ask:

> "How is ontology different from a database schema?"

This is one of the most important conceptual distinctions in semantic engineering.

Traditional databases focus on storing data efficiently.

Ontologies focus on representing meaning formally.

A relational schema might define:

```
Pizza(id, name)
Topping(id, name)
PizzaTopping(pizza_id, topping_id)
```

But the schema itself does not formally express semantic truths such as:

- Vegetarian pizzas cannot contain meat toppings
- Spicy pizzas contain spicy ingredients
- Certain toppings are subclasses of vegetable toppings
- Seafood toppings are disjoint from meat toppings

OWL allows these semantics to be explicitly modeled.

This is why ontologies become foundational for AI and knowledge graph systems.

1.6 Open World Assumption: A New Way of Thinking

One of the most difficult concepts for beginners is the Open World Assumption (OWA).

Traditional databases generally operate under a Closed World Assumption (CWA):

> If something is not stored, it is considered false.

OWL works differently!

Under OWA:

> Absence of knowledge does not imply falsehood.
>
> It just means the knowledge that is unknown yet.

For example:

If an ontology does not specify whether a pizza contains meat, we cannot conclude that it is vegetarian.

This principle aligns much more closely with real-world knowledge representation, where information is often incomplete.

Understanding OWA is essential because it fundamentally changes how semantic systems behave compared to traditional enterprise systems.

Michael DeBellis highlights this repeatedly throughout the tutorial because many modeling errors originate from misunderstanding **open-world reasoning**.

1.7 Reasoners: The Engine Behind Semantic Intelligence

The real magic of ontology engineering appears when we introduce reasoners.

A reasoner is a logic engine that analyzes ontology axioms and derives new knowledge automatically.

For example, if we define:

```
VegetarianPizza ≡ Pizza AND (hasTopping only (CheeseTopping or VegetableTopping)
```

and later create a pizza that only contains cheese and vegetables, the reasoner can automatically classify it as a `VegetarianPizza`.

No manual tagging is required!

This is a profound shift.

The ontology stops being passive documentation and becomes a **reasoning-enabled semantic system**. The reasoner can automatically infer classifications and detect inconsistencies. This is an essential step toward **Executable Knowledge Architecture (EKA)**, but true executability -- where reasoning outcomes automatically trigger actions and workflows -- requires additional layers: **trigger (Θ)** and **execution action (Φ)**, which we will explore in later chapters.

This is precisely the transition EKA describes as moving toward executable intelligence.

1.8 Why Ontology Matters in the AI Era

Modern AI systems increasingly require structured semantic context.

Large Language Models (LLMs) are powerful, but without explicit knowledge structures they often struggle with:

- Consistency
- Explainability
- Governance
- Enterprise semantics
- Controlled reasoning

Poor structures for LLMs normally lead to the "beautiful garbage" to be generated by AI.

Ontology provides the semantic backbone that complements probabilistic AI systems.

This is why ontology engineering is re-emerging as a critical discipline in:

- Knowledge Graphs
- Semantic Search
- AI Agents
- Digital Twins
- Enterprise Knowledge Management
- Data Fabric architectures

The Pizza tutorial may appear simple, but the underlying modeling principles scale into enterprise-grade semantic systems.

1.9 What You Will Build Throughout This Book

As this eBook progresses, we will gradually build increasingly sophisticated ontology capabilities using Protégé.

The journey includes:

- Constructing class hierarchies
- Modeling object properties
- Defining logical restrictions
- Using automated reasoners
- Performing semantic queries
- Applying SWRL rules
- Validating ontologies with SHACL
- Connecting ontology concepts to knowledge graph thinking
- Positioning ontology inside the EKA execution pipeline

By the end, readers should understand not only how to use Protégé, but why ontology engineering matters strategically for modern AI-native enterprises.

1.10 The Bigger Picture

The Pizza ontology is not really about pizza.

It is about learning how to formalize knowledge.

Once you understand:

- classes,
- relationships,
- restrictions,
- inference, and
- semantic reasoning,

you can model almost anything:

- enterprise architectures,
- business capabilities,
- manufacturing systems,
- healthcare vocabularies,
- supply chains,
- digital twins,
- AI agent memory systems, and
- knowledge graphs.

See https://github.com/yasenstar/ArchiMate_Ontology as one practical sample that I'm applying to use Protégé to build ontology for ArchiMate (EA Modeling language).

This is where ontology engineering evolves from academic theory into enterprise intelligence infrastructure.

The future of architecture is not static diagrams.

The future is **reasoning-enabled semantics** that can evolve into **executable knowledge** -- where semantic insights automatically trigger actions, workflows, and intelligent system responses..

And Protégé is one of the gateways into that world.

1.11 Chapter 01 Summary

In this chapter, we established the conceptual and strategic foundations of ontology engineering using the famous `pizza.owl` tutorial as the entry point. Rather than treating Protégé as merely a standalone modeling application (ontology editor), we positioned it as a semantic engineering platform capable of transforming static knowledge into **reasoning-enabled semantic models** -- the essential foundation for **Executable Knowledge Architecture (EKA)**, where triggers and execution actions ultimately operationalize knowledge.

We explored why the Pizza Tutorial became one of the most influential ontology learning examples in the Semantic Web community and examined how Protégé enables users to formally define concepts, relationships, restrictions, and semantic logic through OWL ontologies.

The chapter also introduced the EKA (Executable Knowledge Architecture) perspective, where ontology acts as the semantic bridge between enterprise modeling and knowledge-driven intelligence systems.

Within the EKA implementation roadmap:



the Pizza OWL tutorial specifically focuses on the ontology layer, helping learners understand how formal semantic structures are created and reasoned upon.

In addition, we introduced several core OWL concepts including:

- Classes
- Individuals
- Properties
- Open World Assumption
- Reasoners
- Semantic Inference

These concepts form the foundation for all future ontology engineering activities throughout this book.

Finally, we connected ontology engineering to the broader AI era, explaining why semantic modeling, knowledge graphs, and executable semantics are becoming increasingly important in modern enterprise architecture and intelligence systems.

1.12 Next Chapter Preview

In the next chapter, we will begin working directly inside Protégé and start constructing the foundational structure of the Pizza ontology.

Readers will learn how to:

- Create ontology projects
- Understand Protégé workspace components
- Build class hierarchies
- Create subclasses
- Define semantic categories
- Organize ontology structures properly

This marks the beginning of hands-on ontology engineering using Protégé.

The transition from theory to implementation starts there.

1.13 Reference

- Michael DeBellis, A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins
- Protégé official resources and tutorials
- Protégé Pizza Ontology learning repository: https://github.com/yasenstar/protege_pizza
- EKA Official Website: https://xiaoqi.com

1.14 Key Concepts

Term	Definition
Ontology	A formal, machine-readable model that defines concepts, relationships, constraints, and inference logic within a domain.
OWL (Web Ontology Language)	A W3C standard language for representing semantic knowledge in a logically rigorous, machine-interpretable form.
Protégé	An open-source semantic modeling platform and ontology editor used to build, reason over, and validate OWL ontologies.
Class	A category or type of thing in an ontology (e.g., Pizza, CheeseTopping).
Individual	A concrete instance of a class (e.g., MargheritaPizza, MozzarellaTopping).
Object Property	A relationship that connects one individual to another (e.g., hasTopping).
Data Property	A relationship that connects an individual to a literal value (e.g., hasCalories).
Reasoner	A logic engine that checks consistency, infers implicit relationships, and automatically classifies individuals based on defined axioms.
Open World Assumption (OWA)	The principle that absence of information does not imply falsity; knowledge may be incomplete.
Closed World Assumption (CWA)	The principle that anything not explicitly known is assumed false (typical of relational databases).
Executable Knowledge Architecture (EKA)	An architectural framework that transforms formal knowledge into machine-executable intelligence through a structured pipeline of diagrams, meta-models, ontologies, knowledge graphs, and executable intelligence.
Semantic Inference	The process by which a reasoner derives new knowledge from existing axioms without manual intervention.

1.15 One More Thing...

Small tip, how to type `Protégé` in Markdown with those French `é`?

Answer: 😉, just using your keyboard's number pad, hold `Alt` key and key in `0233` or `233`, like `Alt + 0233` or `Alt + 233`, then you see the magic. (This is called Alt Code.) Enjoy!

> **Note**
>
> The Alt Code is Windows only. This should be noted. Mac OS and Linux have their own methods for creating diacritical characters. (*Thanks for pointed out by @sarcanon, refer to issue log #43*)

Last updated at: 2026-09-12